CSC 648 (Section 3)

Vibe Check

ARKHA (Team 6)

April 8th, 2026

Harry Kakadiya (Team Lead): hkakadiya@sfsu.edu
Kaitlyn Ashby (Github master, Technical writer): kashby@sfsu.edu
Rahul Srinath (Software Architect): rsrinath@sfsu.edu Amulya
Sagi (Backend Lead): asagi@sfsu.edu
Aljhay Soriano (Scrum Master): asoriano6@sfsu.edu

https://github.com/sfsu-joseo/csc648-848-project-sp26-ARKHA

Revision	Description
7	Added Figma Wireframes, documented creational design patterns for M3V1 draft
6	Added Network diagram per feedback, and documented API/Algorithms for M2V2 submission
5	Created ERD, UML diagram and scalability diagram for M2V1 submission
4	Refined data definitions, starting to prioritize functional requirements, pen + paper mockups created M2V1 draft
3	Added 70 more Functional Requirements per feedback, added use cases for other features, friends, badges, push notifications, added data definitions M1V2 submission
2	Added 40 Functional Requirements, and main use cases (navigating heap map, posting notes) M1V1 submission
1	Created Vibe Check Shared Documentation, submitted tech stack for approval M1V1 draft

Table of Contents

Data Definitions

User

A User is a person who uses the application to view nearby locations and interact with content. A User can create Notes and can reply to or react to existing Notes.

Attribute	Description
user_id	Unique identifier for each User. Primary Key.
username	Public display name shown on Notes and the User Profile.
email	Email address used for login and system notifications.
password	Plaintext version of password that the registered user signs in with, authenticated against password_hash.
password_hash	Securely hashed password. Never stored in plaintext.
profile_picture_url	URL pointing to the user's profile photo.
created_at	Timestamp of when the account was created.
account_status	Current state of the account: active, suspended, or deleted.

Registered User

A Registered User is a User who has completed the full account registration process by verifying their email and setting a password. Only Registered Users are permitted to create Notes, post Replies, submit Reactions, and access social features such as Friends and Badges. A guest or unverified User becomes a Registered User upon successful email verification. This is a logical specialization of the User entity and is tracked through the email_verified and account_status fields on the User.

Attribute	Description
reg_user_id	Unique identifier for the Registered User record. Primary Key.
user_id	Foreign key linking to the base User entity. Each Registered User corresponds to exactly one User.
email_verified	Boolean flag indicating whether the user has confirmed their email address. Must be true for full platform access.
registration_completed_at	Timestamp of when the user completed the full registration process.
role	The permission level of the registered user: standard or admin. Default is standard.

Location

A Location represents a physical place shown on the map (e.g., café, library, venue, park). Notes are tied to a specific Location to ensure that updates are relevant to that place.

Attribute	Description
location_id	Unique identifier for each Location. Primary Key.
name	Display name of the location (e.g., J. Paul Leonard Library, Philz Coffee).
address	Human-readable street address used for display and search.
latitude	GPS latitude coordinate. Used for map rendering and radius enforcement.

longitude	GPS longitude coordinate. Used for map rendering and radius enforcement.
category_tags	One or more category labels assigned to this location (e.g., Restaurant, Study Spot, Venue, Park).
avg_vibe_score	(Derived) Rolling average of Vibe Scores calculated from non-expired Notes at this location. Recomputed by the system when Notes are added or expire. Never stored directly.

Note

A Note is a short, location-locked post created by a User to describe what is happening at a Location in real time. Each Note has a timestamp and an expiration time so that outdated information is automatically removed.

Attribute	Description
note_id	Unique identifier for each Note. Primary Key.
user_id	Foreign key linking to the User who posted the Note.
location_id	Foreign key linking to the Location this Note describes.
content	The text body of the Note. Maximum 280 characters.
vibe_score	The atmosphere rating selected by the user at the time of posting: Dead, Quiet, Moderate, Busy, or Buzzing. Stored as an attribute of the Note, not as a separate entity. See Vibe Score Values table below.
is_anonymous	Boolean flag indicating whether the poster's username is hidden from other users. Default is false.
created_at	Timestamp of when the Note was posted.

expires_at	Timestamp of when the Note is automatically removed by the system. Typically 2–4 hours after created_at.

Vibe Score Values:

Numeric Value	Label	Description
1	Dead	No activity, completely empty
2	Quiet	Low activity, few people present
3	Moderate	Average activity, some people present
4	Busy	High activity, noticeably crowded
5	Buzzing	Maximum activity, very crowded or lively

Category Tag

A Category Tag is a label used to classify a Note or Location (e.g., Restaurant, Study Spot, Venue, Parking). Category Tags allow users to filter and view relevant content more efficiently.

Attribute	Description
tag_id	Unique identifier for each Category Tag. Primary Key.
tag_name	Display label for the tag (e.g., Restaurant, Study Spot, Venue, Parking, Park).

Reply

A Reply is a response to a specific Note that provides additional context or updated information. Each Reply belongs to one Note.

Attribute	Description

reply_id	Unique identifier for each Reply. Primary Key.
note_id	Foreign key linking to the Note this Reply belongs to.
user_id	Foreign key linking to the User who posted the Reply.
content	Text content of the Reply. Maximum 280 characters.
created_at	Timestamp of when the Reply was posted.

Reaction

Definition: A Reaction represents a quick feedback action (an upvote) that a User can apply to a Note. Reactions help indicate which Notes are useful or accurate. Each User may submit at most one Reaction per Note.

Attribute	Description
reaction_id	Unique identifier for each Reaction. Primary Key.
note_id	Foreign key linking to the Note being reacted to.
user_id	Foreign key linking to the User who submitted the Reaction.
created_at	Timestamp of when the Reaction was submitted.

Heatmap Activity

Heatmap Activity is a visual representation of recent activity in a geographic area. It is generated from recent Notes and user interactions within a specific time window and helps users quickly assess how active an area is. All values in this entity are derived from existing Notes and are never stored directly.

Attribute	Description

latitude	Center GPS latitude of the heatmap area cell.
longitude	Center GPS longitude of the heatmap area cell.
activity_score	(Derived) Count of non-expired Notes in this area within the active time window. Computed dynamically by the system. Higher score = more intense heat color on the map.
time_window	The rolling time range used to compute activity. Default is 2 hours.
computed_at	Timestamp of when this heatmap snapshot was last calculated by the system.

User Profile

A User Profile stores user preferences and contribution statistics. Each User has exactly one Profile. Preferences include notification settings, default search radius, and preferred categories. Contribution statistics are used by the gamification system to track badge progress.

Attribute	Description
profile_id	Unique identifier for each User Profile. Primary Key.
user_id	Foreign key linking to the User this Profile belongs to. Each User has exactly one Profile.
default_radius_km	The default search and filter radius in kilometers. Default is 1.0 km.
preferred_categories	List of category tags the user prefers to see by default on the map.

visibility	Controls who can view this profile: public or friends_only.
total_notes_posted	Lifetime count of Notes posted by this user. Used for badge threshold tracking.
total_reactions_received	Lifetime count of upvotes received on this user's Notes. Used for badge threshold tracking.
friend_notifications_enabled	Boolean flag indicating whether the user receives notifications when a friend leaves a nearby Note. Default is true.
notifications_enabled	Boolean flag indicating whether the user receives general system notifications. Default is true.

Notification

Definition: A Notification is a system-generated alert sent to a User when a relevant event occurs, such as a nearby trending location, a friend leaving a Note nearby, or a badge being earned.

Attribute	Description
notification_id	Unique identifier for each Notification. Primary Key.
user_id	Foreign key linking to the User receiving this Notification.
type	The category of triggering event: trending_location, friend_note, or badge_earned.
message	Human-readable notification text displayed to the user.
is_read	Boolean flag indicating whether the user has viewed this notification. Default is false.

created_at	Timestamp of when the Notification was generated by the system.

Friend Connection

A Friend Connection represents a mutual relationship between two Users. It allows Users to see each other's Notes highlighted on the map and receive notifications when a friend leaves a Note nearby. A connection is only considered active when its status is accepted.

Attribute	Description
friendship_id	Unique identifier for each Friend Connection. Primary Key.
user_id_1	Foreign key linking to one side of the friendship.
user_id_2	Foreign key linking to the other side of the friendship.
status	Current state of the connection: pending, accepted, or blocked.
created_at	Timestamp of when the friend request was originally sent.

Badge

A Badge is an achievement awarded to a User for meeting specific contribution milestones, such as leaving 10 Notes, visiting 5 cafés, or receiving 25 reactions. Badges are awarded automatically by the system when a User meets the defined threshold.

Attribute	Description
badge_id	Unique identifier for each Badge. Primary Key.
badge_name	Display name of the badge shown on the user's profile (e.g., Coffee Connoisseur, Local Legend).

description	Human-readable explanation of what the user must do to earn this badge.
requirement_threshold	The numeric count required to earn this badge (e.g., 10 notes posted at coffee shops).

User Badge

A User Badge is a junction entity that records the relationship between a User and a Badge they have earned. It is created automatically by the system when a User meets the contribution threshold for a given Badge. A User can earn many Badges, and a Badge can be earned by many Users.

Attribute	Description
user_badge_id	Unique identifier for this earned badge record. Primary Key.
user_id	Foreign key linking to the User who earned the badge.
badge_id	Foreign key linking to the Badge that was earned.
earned_at	Timestamp of when the badge was awarded to the user by the system.

Functional Requirements

Priority 1

Authentication

- FR 1.1: The user shall be able to create an account with an email and password.
- FR 1.2: The user shall be able to log in and log out.
- FR 1.3: The user shall be required to be authenticated in order to create notes, replies, or reactions.
- FR 1.4: The system shall send a verification email to the user upon account creation.
- FR 1.5: The system shall grant full platform access only after the user's email address has been verified.
- FR 1.6: The system shall display the user's registration status on their profile.

- FR 1.7: The system shall allow a user to resend the verification email if the original was not received.

Location

- FR 2.1: The user shall be able to grant permission for the app to access their current location.
- FR 2.2: The user shall be able to view nearby locations on the map.
- FR 2.3: The user shall be able to select a specific location from the map.

Heatmap

- FR 3.1: The user shall be able to view a heatmap showing activity in the nearby area.
- FR 3.3: The user shall be able to zoom in and out of the map.

Notes

- FR 4.1: The user shall be able to view notes related to a specific location.
- FR 4.7: The system shall enforce a limit of one reaction per user per note
- FR 8.9: The system shall hide notes and replies posted by blocked users from the blocking user's view.
- FR 4.9: The user shall be able to post a note at a specific location.
- FR 4.10: The user shall be able to compose text content when creating a note.
- FR 4.13: The system shall restrict note posting to users who are within the defined geographic radius of a location.
- FR 4.14: The system shall display a descriptive error message to the user when a note fails to post.
- FR 4.15: The user shall be able to view notes in a card format resembling a letterboxed review, showing vibe level, note content, timestamp, reaction count, and reply count.
- FR 4.17: The user shall be able to view the username or anonymous handle associated with each note.
- FR 4.18: The user shall be able to view the time elapsed since a note was posted.

Vibe Score

- FR 7.1: The user shall be able to select a vibe level (Dead, Quiet, Moderate, Busy, Buzzing) when creating a note.
- FR 7.2: The user shall be able to view the aggregate vibe score of a location based only on non-expired notes.
- FR 7.3: The system shall display a fallback message indicating limited data availability when fewer than the minimum required notes exist to calculate a vibe score.
- FR 7.4: The user shall be able to view vibe scores visually using descriptive labels rather than numerical star ratings.

- FR 7.5: The user shall be able to view an updated vibe score for a location when new notes are added or expired notes are removed.

Priority 2

Heatmap

- FR 3.2: The user shall be able to filter the heatmap by category.

Notes

- FR 4.2: The user shall be able to view replies to a note.
- FR 4.3: The user shall be able to reply to notes posted by other users.
- FR 4.4: The user shall be able to delete their own reply.
- FR 4.5: The user shall be able to view reactions to a note.
- FR 4.6: The user shall be able to react to notes posted by other users.
- FR 4.7: The user shall be able to react only once to the same note.
- FR 4.8: The user shall be able to remove a reaction.
- FR 4.11: The user shall be able to edit their own note.
- FR 4.12: The user shall be able to delete their own note.
- FR 4.16: The user shall be able to visually distinguish between notes posted by friends and notes posted by other users on the map.
- FR 4.19: The user shall be able to view the remaining time before a note expires.

Search/Filter

- FR 6.1: The user shall be able to search for locations by name or category.
- FR 6.2: The user shall be able to filter search results to a radius.
- FR 6.3: The user shall be able to filter search results by vibe level.

Vibe Score

- FR 7.6: The user shall be able to filter locations on the map by vibe level.

Moderation

- FR 11.1: The user shall be able to report a note for inappropriate content.
- FR 11.2: The user shall be able to provide a reason for reporting a note.
- FR 11.3: The user shall be able to report a reply for inappropriate content.
- FR 11.4: The user shall be able to provide a reason for reporting a reply.

User

- FR 10.1: The user shall be able to edit their username.

- FR 10.2: The user shall be notified if the username they want to change to is unavailable.
- FR 10.3: The user shall be able to change their password.
- FR 10.4: The user
- FR 10.5: The user shall be able to view their posting history.
- FR 10.6: The user shall be able to view another user's public profile
- FR 10.7: The user shall be able to view the username associated with notes, replies and reactions posted by other users.
- FR 10.8: The user shall be able to view their public user profile.

Notification

- FR 5.1: The user shall be able to receive a notification when a trendy location is nearby.
- FR 5.2: The user shall be able to enable or disable notifications in settings.

Priority 3

Friends

- FR 8.1: The user shall be able to send a friend request to another registered user.
- FR 8.2: The user shall be able to accept or decline an incoming friend request.
- FR 8.3: The user shall be able to remove a friend from their friends list.
- FR 8.4: The user shall be able to view a list of their current friends.
- FR 8.5: The user shall be able to see notes left by their friends on the map, visually distinguished from notes by non-friends.
- FR 8.6: The user shall be notified when a friend leaves a note at a nearby location.
- FR 8.7: The user shall be able to enable or disable friend activity notifications in settings.
- FR 8.8: The user shall be able to block another user, preventing them from viewing their notes or sending friend requests.
- FR 8.9: The user shall be able to view notes posted by only unblocked users.
- FR 8.10: The user shall be able to unblock a previously blocked user.
- FR 8.11: The user shall be able to see an indicator icon on map pins where a friend has recently left a note.

Badge

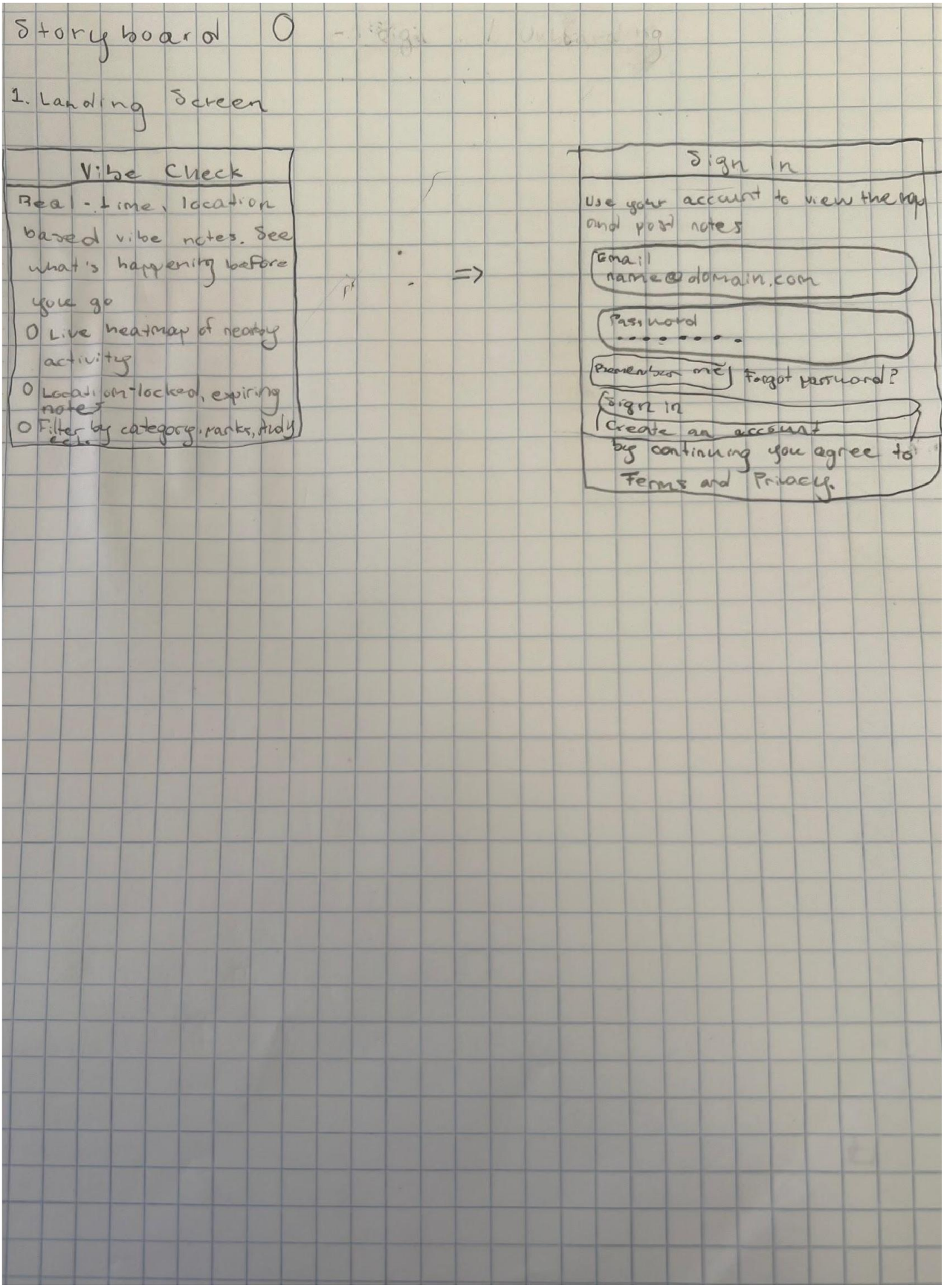
- FR 9.1: The user shall be automatically awarded badges when they meet defined contribution thresholds.
- FR 9.2: The user shall be able to view all badges they have earned on their profile.
- FR 9.3: The user shall be able to view all available badges and the requirements to earn them.
- FR 9.4: The user shall be notified when they earn a new badge.
- FR 9.5: The user shall be able to track the number of notes they have posted toward badge thresholds.

- FR 9.6: The user shall be able to track the number of unique location types (e.g., cafes, venues, parks) they have posted notes at.
- FR 9.7: The user shall be able to view the number of reactions their notes have received toward badge thresholds.
- FR 9.8: The user shall be able to display earned badges on their public profile. **User**

- FR 10.4: The user shall be able to upload a profile picture.
- FR 10.5: The user shall be able to delete their account.

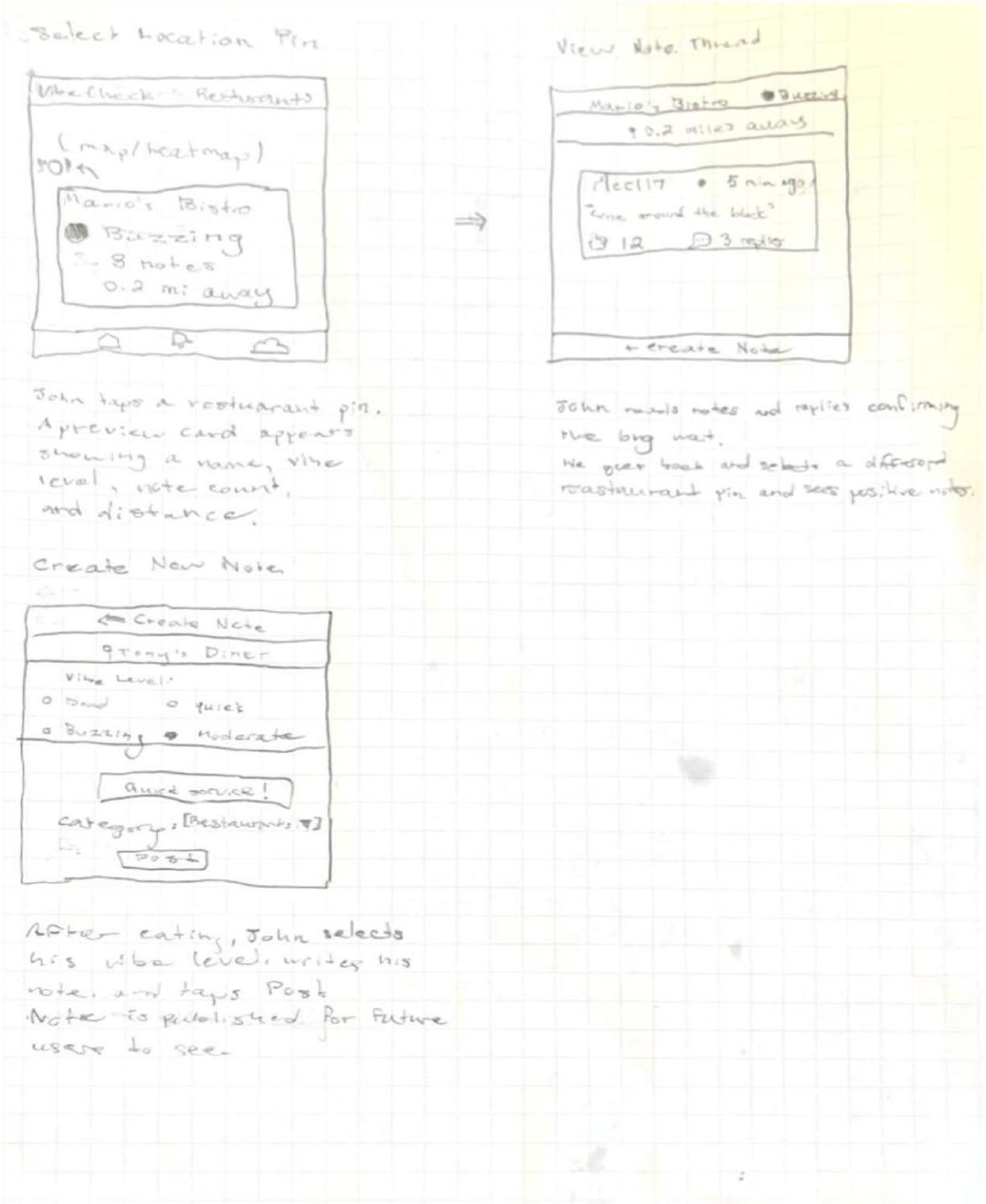
Mockups and Storyboards

Storyboard 0: Sign in/On boarding



A user with an associated user_id can sign into the Vibe Check system with their email and password.

Storyboard 1: Exploring nearby restaurants



Storyboard 1.

Get current location

John allows location access.

View Heatmap

red = busy
Yellow = mild
green = nothing?

Heatmap loads showing activity levels nearby.

Filter by Category

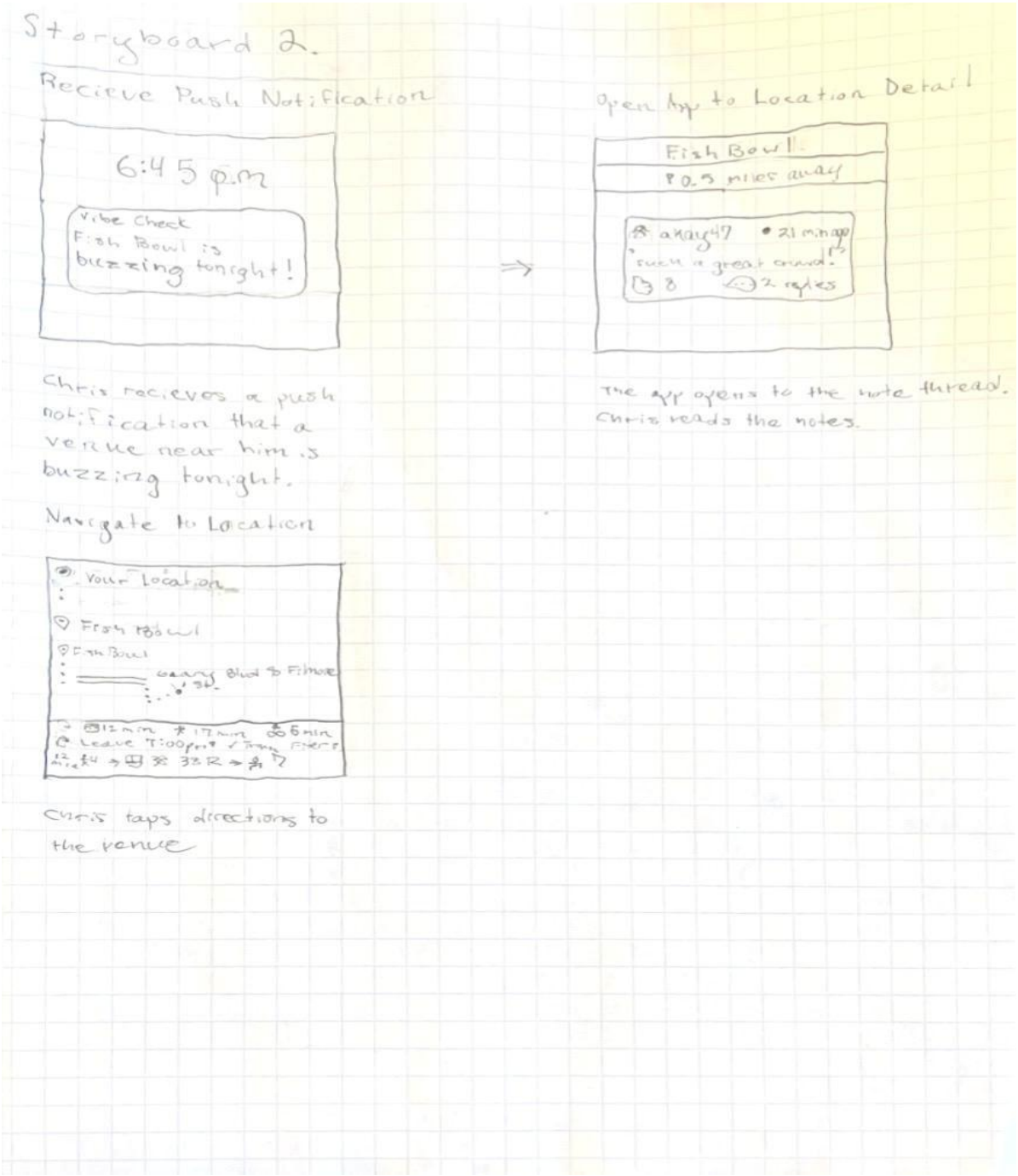
John taps Filter icon.
Selects "Restaurants".
Taps Apply.

View Nearby Location Pins

Map now shows only restaurant pins in the area.
John browses nearby options

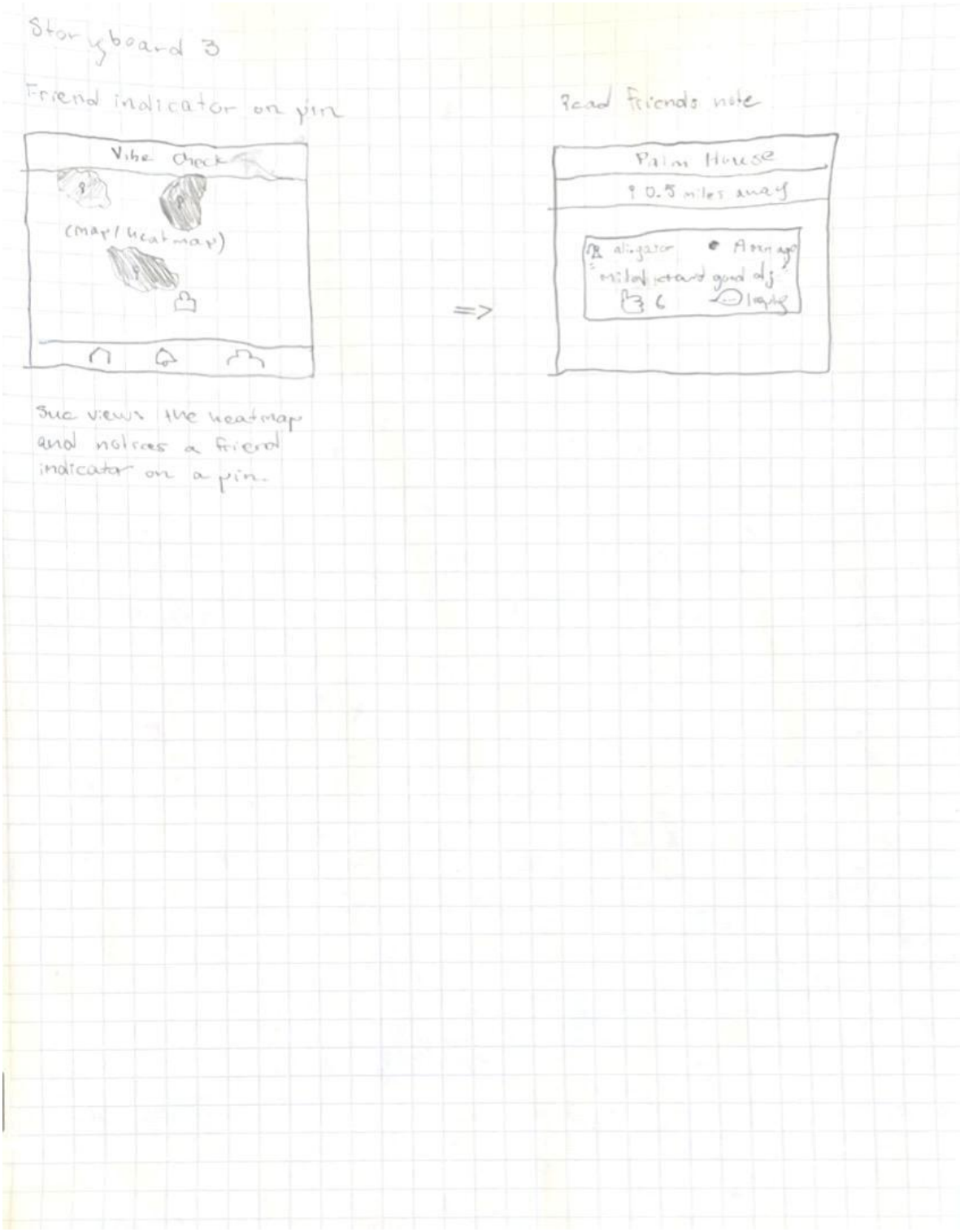
use the heatmap to explore various locations. In addition, they can filter the heatmap to only show certain locations such as restaurants.

Storyboard 2: Push notifications



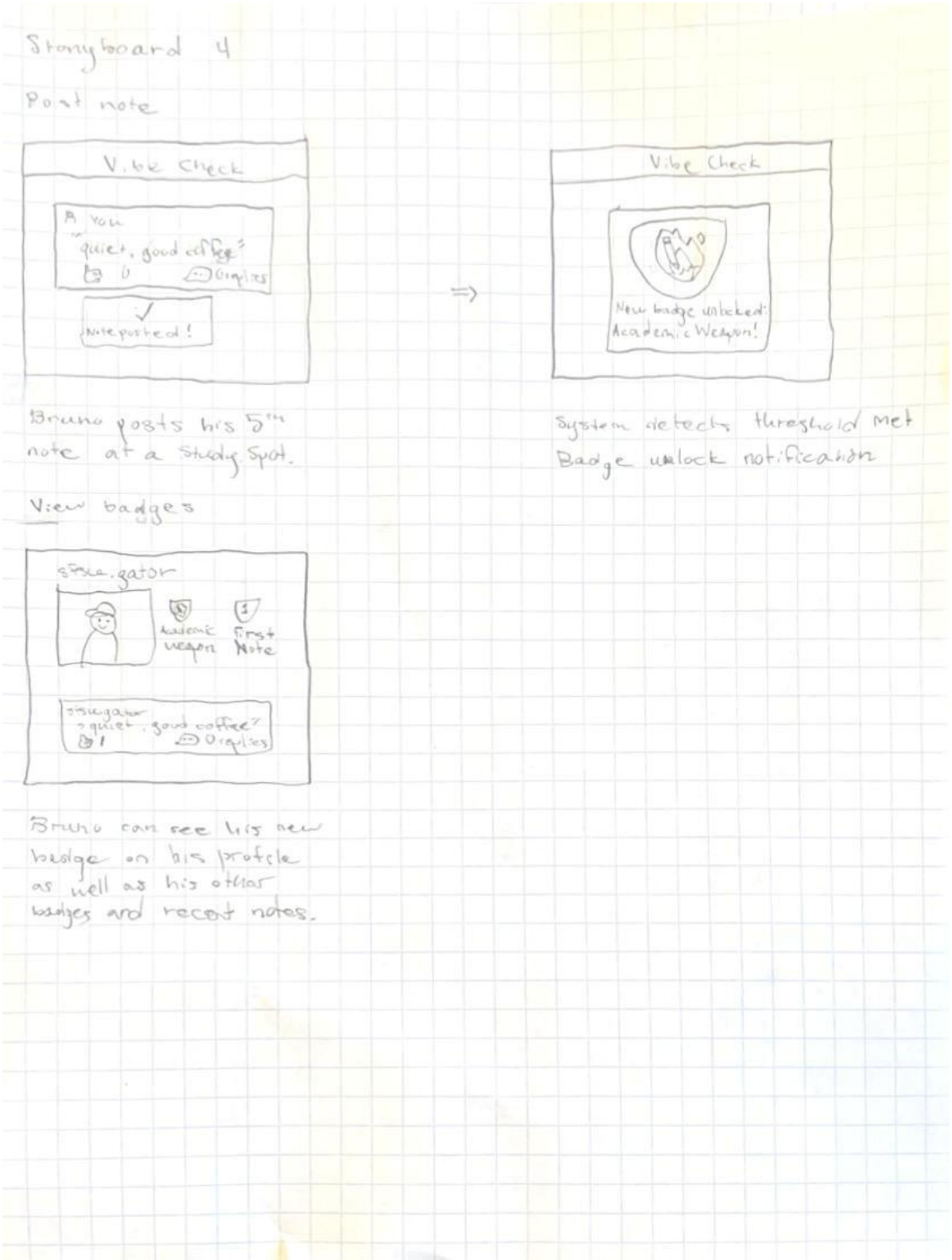
Users can get push notifications sent to their device letting them know if a location near them is buzzing.

Storyboard 3: Friends

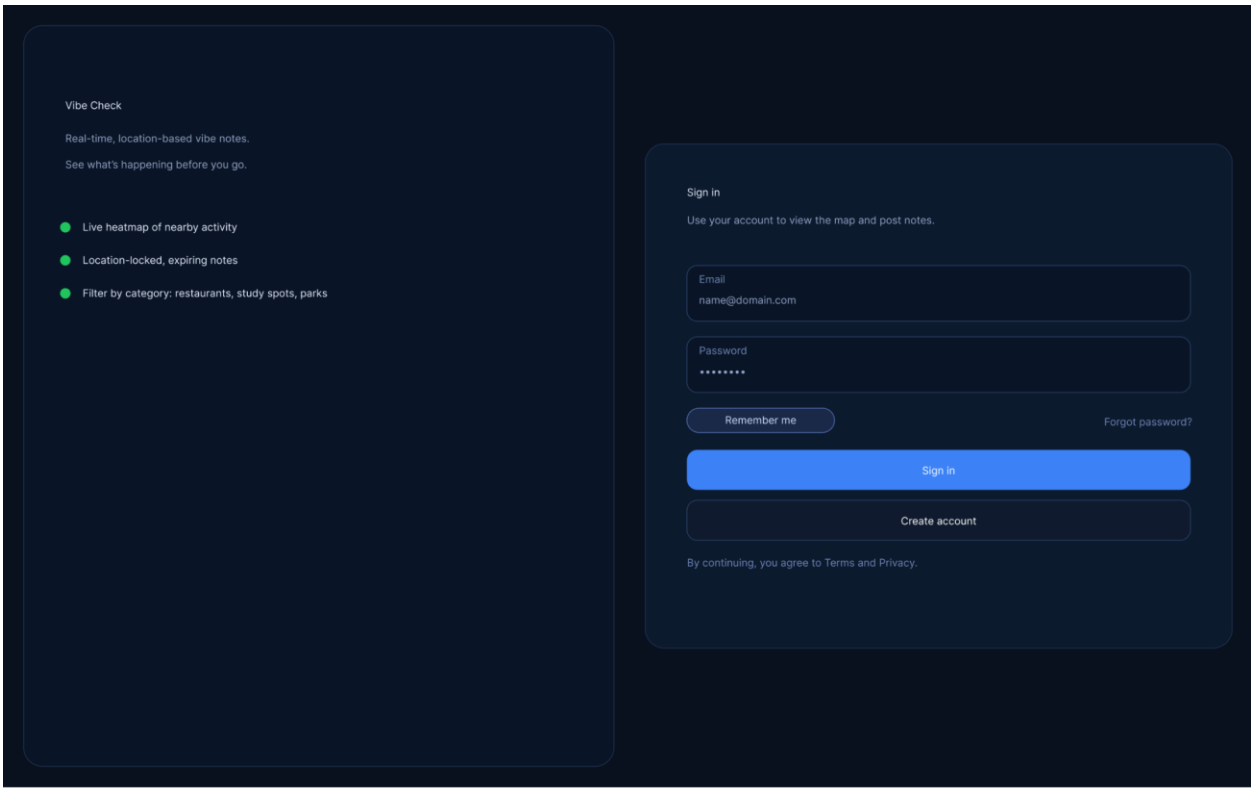


Users can see their friends on the heatmap. Users can also see their friends' notes by clicking their icon on the map.

Storyboard 4: Badges



receive different badges the more they interact with the map.



High Level System Design

Database Architecture

Initial database requirements driven by priority 1 needs

- **users** — FR 1.1, 1.2, 1.4–1.7. The ERD's account_status attribute handles FR 1.6 (display registration status). email_verified (boolean for FR 1.5), verification_token (needed to send and re-validate the link, FR 1.4/1.7), and verification_sent_at / verified_at (timestamps to track resend eligibility and status display).
- **user_profiles** — FR 1.6. The ERD has a separate user_profiles entity in a 1-to-1 relationship with users. For Priority 1, the relevant field is visibility, which can carry registration/verification status display logic. The default_radius_km and notifications_enabled fields are present in the ERD and kept here even though they activate more fully in later priorities.
- **locations** — FR 2.2, 2.3, 4.13. The ERD defines lat/lng, name, address, and category_tags. A radius_meters field is added to support the geographic posting restriction check (FR 4.13), since the ERD's avg_vibe_score is a *derived* attribute (shown in italics) and not stored.

- **notes** — FR 4.1, 4.9, 4.10, 4.13–4.15, 4.17–4.18, 7.1, 7.2, 7.5. The ERD defines content, vibe_score (renamed to vibe_level in the relational model to store the enum: Dead / Quiet / Moderate / Busy / Buzzing per FR 7.1), is_anonymous, created_at, and expires_at. The expires_at column is the key driver for FR 7.2 and 7.5 — the aggregate vibe score queries must filter WHERE expires_at > NOW(). is_anonymous combined with a join back to users.username drives FR 4.17.
- **reactions** — FR 4.7. The ERD models this as a received relationship between notes and reactions. A UNIQUE(user_id, note_id) constraint enforces the one-reaction-per-user rule at the database level. The ERD shows only reaction_id and created_at as attributes, which is correct for Priority 1 (no reaction type/emoji needed yet).
- **replies** — FR 4.15 (reply count on note cards), FR 1.3 (auth required). The ERD's has relationship between notes and replies maps cleanly. is_anonymous is added to mirror the same identity-display logic as notes (FR 4.17).
- **blocks** — FR 8.9. Not explicitly modeled in the ERD, but required by Priority 1. This is a simple self-referencing join on users with blocker_id and blocked_id. Filtered at query time: WHERE notes.user_id NOT IN (SELECT blocked_id FROM blocks WHERE blocker_id = ?).

Key constraints to define Data Definitions

- UNIQUE(user_id, note_id) on reactions — enforces FR 4.7
- UNIQUE(blocker_id, blocked_id) on blocks
- CHECK vibe_level IN ('dead','quiet','moderate','busy','buzzing') on notes
- INDEX(location_id, expires_at) on notes — drives efficient vibe score aggregation (FR 7.2/7.5) and heatmap queries (FR 3.1)
- avg_vibe_score on locations stays a *derived* attribute as the ERD intends — computed via query, not stored

PostgreSQL 15.x with PostGIS

PostgreSQL 15 is an open-source relational DBMS for Vibe Check's needs.

PostGIS is a spatial extension that adds native geographic object support directly into PostgreSQL. Rather than storing latitude and longitude as plain floats, PostGIS lets you store them as a true GEOMETRY or GEOGRAPHY column and run spatial queries using indexed operations. For Vibe Check this matters in three concrete places:

- **FR 4.13 — geographic radius enforcement.** Instead of doing float math in application code, PostGIS lets you write ST_DWithin(user_location, location_point, radius_meters) directly in the query. The check runs at the database layer, making it harder to bypass.
- **FR 2.2 / 3.1 — nearby locations and heatmap.** ST_DWithin and ST_Distance efficiently find all locations within a user's visible area. Without PostGIS this requires a bounding-box approximation or expensive full-table scans.
- **Indexing.** PostGIS uses a **GiST index** on geometry columns, which supports fast bounding-box lookups. This is what makes spatial queries scale — a plain B-tree index on float lat/lng columns cannot do the same job.

In practice, the locations table's lat/lng column becomes a single GEOGRAPHY(POINT, 4326) column (SRID 4326 = WGS 84, the same coordinate system used by GPS), and every spatial check runs through PostGIS rather than application-level arithmetic. The rest of the schema — users, notes, reactions, replies, blocks — uses standard PostgreSQL types with no PostGIS dependency.

Decision on Media Storage Strategy - AWS S3 + Cloudfront

This is for image storage Organize by location ID (e.g. images/{location_id}/{photo_id}.jpg), keep the bucket private, and serve
everything through CloudFront with Origin Access Control. Gives caching pre-signed URL support for uploads, and lifecycle rules to transition old photos to cheaper storage tiers automatically.

Backend Architecture

Architectural Summary

Reliability

Vibe Check's reliability strategy is built around avoiding single points of failure at every layer. PostgreSQL 15 is deployed with a **primary/replica setup** — the primary handles all writes while one or more read replicas serve read-heavy queries like fetching notes for a location or computing vibe scores. If the primary becomes unavailable, a replica can be promoted with minimal downtime.

I need to check this out, but connection pooling via **PgBouncer** sits between the application and the database, preventing connection exhaustion under traffic spikes — a particular concern given

that note feeds and heatmap queries can burst when many users check the same location simultaneously.

At the application layer, the API is deployed as **multiple stateless instances** behind a load balancer. Stateless means no session data lives on the server — authentication tokens, user state, and request context are all carried in the request itself (JWT tokens) or retrieved from the database. Any instance can handle any request, so losing one instance has no user-facing impact. The load balancer performs health checks and automatically stops routing to unhealthy instances.

TBD: For the S3/CloudFront media layer, AWS's own infrastructure provides regional redundancy. CloudFront's edge caching means a brief S3 outage does not immediately surface to users whose requested files are already cached at the edge. This is also not really needed yet for our priority 1 listing.

Note expiry (required for FR 7.2) is handled by a **background worker** that periodically marks or deletes expired notes rather than relying on query-time filtering alone. This keeps the vibe score aggregation queries fast and ensures expired content is not surfaced even if a cache or query edge case is hit.

Security

Security is enforced in layers from the network edge down to the row level.

Authentication uses short-lived **JWT access tokens** paired with longer-lived refresh tokens. The access token carries the user_id and email_verified status, allowing middleware to gate any write operation (notes, replies, reactions) without a database round-trip on every request. Refresh tokens are stored server-side and can be revoked, which plain stateless JWTs alone cannot support.

Email verification (FR 1.5) is enforced at the API middleware level — any request to create a note, reply, or reaction checks the email_verified claim in the token before proceeding. Unverified users can read but not write.

Geographic enforcement (FR 4.13) runs at the database layer using PostGIS's ST_DWithin function rather than in application code. This means the restriction cannot be bypassed by a manipulated API request — the spatial check is part of the query itself.

Row-level visibility for blocked users (FR 8.9) is applied as a query filter on every note and reply fetch: WHERE user_id NOT IN (SELECT blocked_id FROM blocks WHERE blocker_id = $current_user). This is enforced server-side and never depends on the client omitting blocked content.

The **S3 bucket is fully private**. No object is publicly accessible directly. All media delivery goes through CloudFront, which presents a signed origin request to S3 using an **Origin Access Control (OAC)** policy. This means even if a direct S3 URL is known, it returns a 403. Presigned upload URLs expire after five minutes and are scoped to a single object key.

Environment secrets — database credentials, JWT signing keys, AWS credentials — are never hardcoded. They are injected at runtime via environment variables managed through a secrets manager (AWS Secrets Manager or similar), keeping them out of source control and container images entirely.

Containers

The application is containerised using **Docker**, with each service packaged as an independent image. For Priority 1 the container surface is deliberately minimal:

The **API service** is a Node.js or equivalent backend container exposing the REST endpoints. It is stateless by design — no local disk writes, no in-memory session state — which means horizontal scaling is a matter of increasing the replica count with no coordination required between instances.

The **background worker** runs as a separate container on a scheduled basis, handling note expiry and any future async jobs. Keeping it separate from the API container means a worker failure or slow job does not affect API response times.

Docker Compose is used for local development, wiring the API container, worker container, and a local PostgreSQL instance together. In production, containers are orchestrated by **AWS ECS (Elastic Container Service)**, which handles deployment, health checks, rolling updates, and auto-scaling without requiring manual server management.

Container images are built in a **CI/CD pipeline** (GitHub Actions). Images are never deployed from a developer's local machine — every production deployment goes through the pipeline, which runs tests, builds the image, pushes it to a registry (Amazon ECR), and triggers a rolling deployment that replaces old containers with new ones with zero downtime.

Consistency

Vibe Check has two areas where consistency requires deliberate design rather than defaults.

Vibe score consistency (FR 7.2, 7.5) is the most sensitive. The aggregate vibe score for a location is computed from non-expired notes, and it must update when new notes are posted or old ones expire. The chosen approach is **read-time computation with a database index** — the

score is not cached or materialised, it is computed fresh on each request using the INDEX(location_id, expires_at) index on the notes table. This guarantees the score always reflects current data with no risk of a stale cached value being shown. The avg_vibe_score derived attribute in the ERD is intentionally not stored as a column for this reason. If query performance degrades at scale, a short-lived application-level cache (Redis, TTL of 60 seconds) can be introduced without changing the consistency model — the database remains the source of truth.

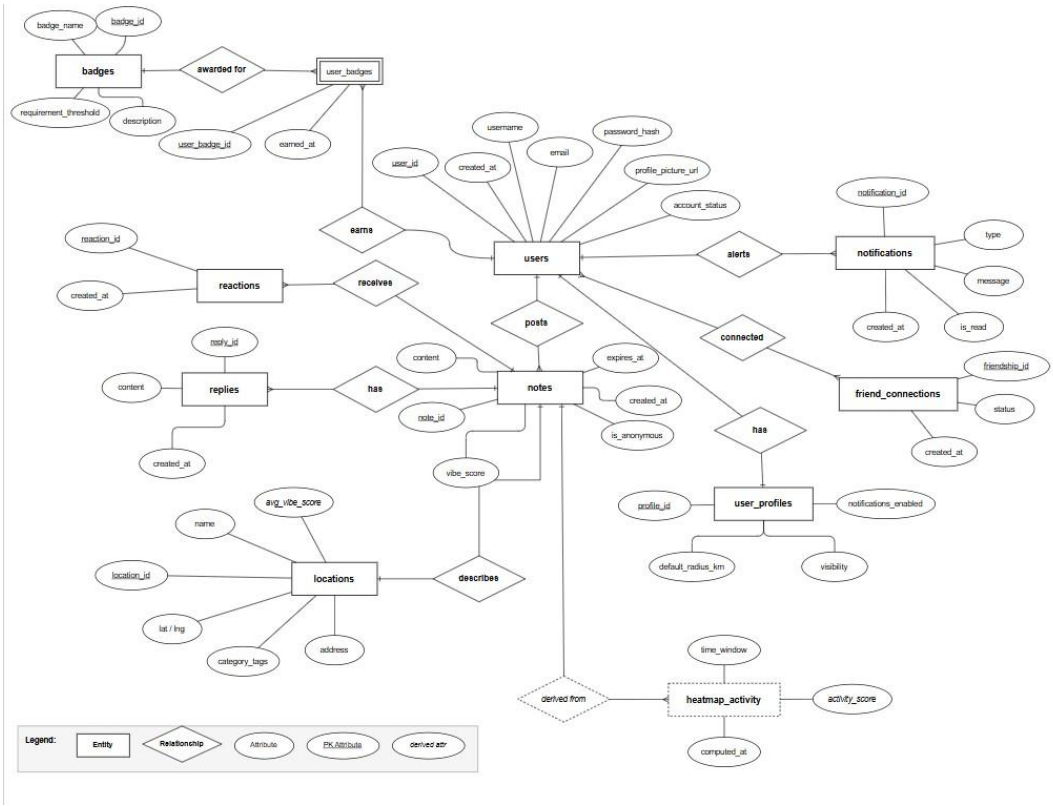
Reaction uniqueness (FR 4.7) is enforced by a UNIQUE(user_id, note_id) constraint at the database level. The application does not need to check for an existing reaction before inserting — it attempts the insert and handles the unique constraint violation as a known, expected error. This is the strongest possible consistency guarantee: two simultaneous requests from the same user cannot both succeed, regardless of race conditions at the application layer.

Note posting (FR 4.9, 4.13) uses a **single database transaction** that includes the PostGIS spatial check and the note insert together. Either both succeed or neither does — there is no window where the spatial check passes but the insert fails silently, or where a note is written before the geographic validation completes.

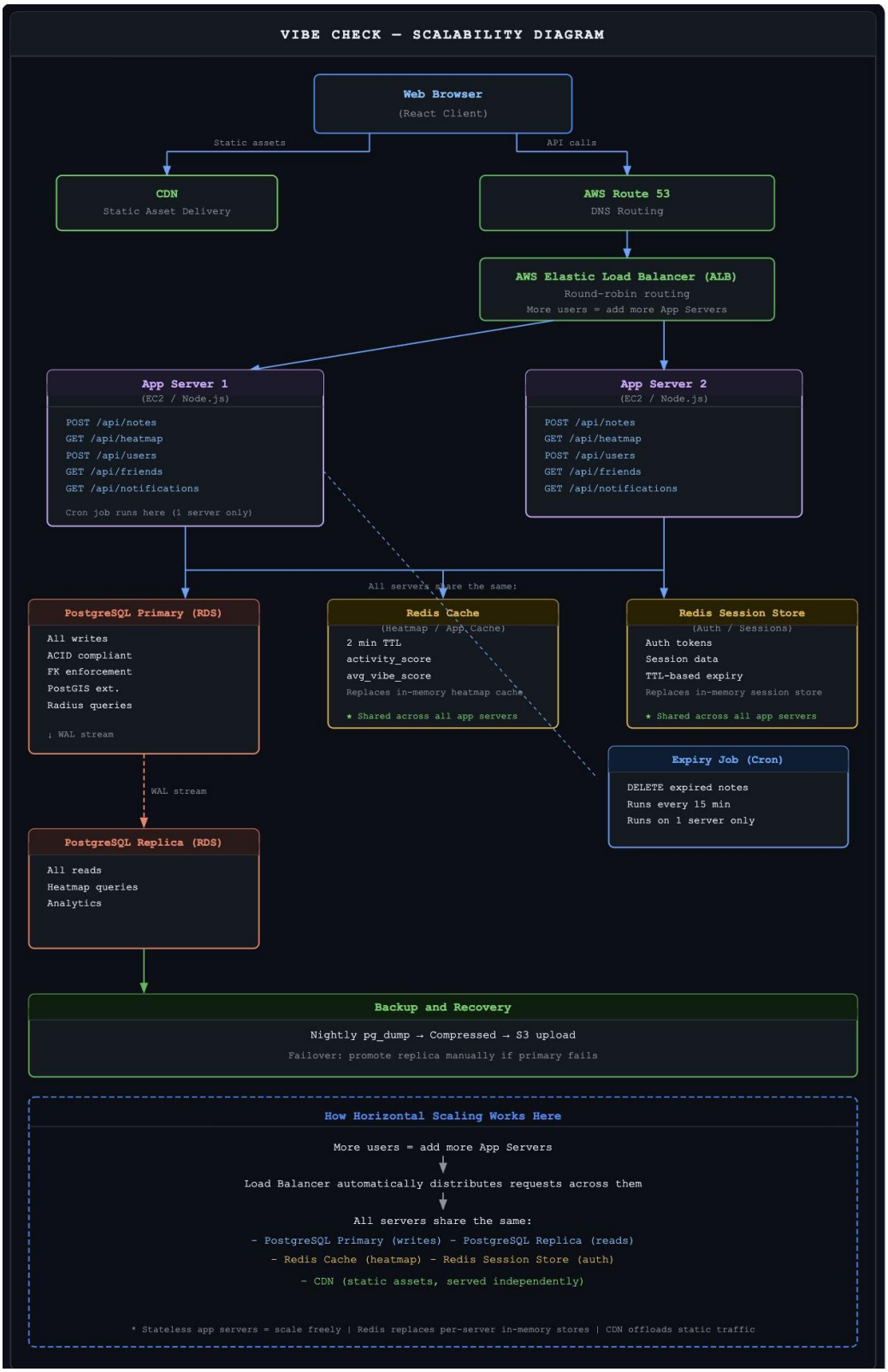
Justification of design patterns used

Design patterns are justified through the ERD as well ease of use for geo-spacial data, secure online trafficking, on time postings, post deletions, account management, cloud management and database functionality based off of project concerns and functional requirements.

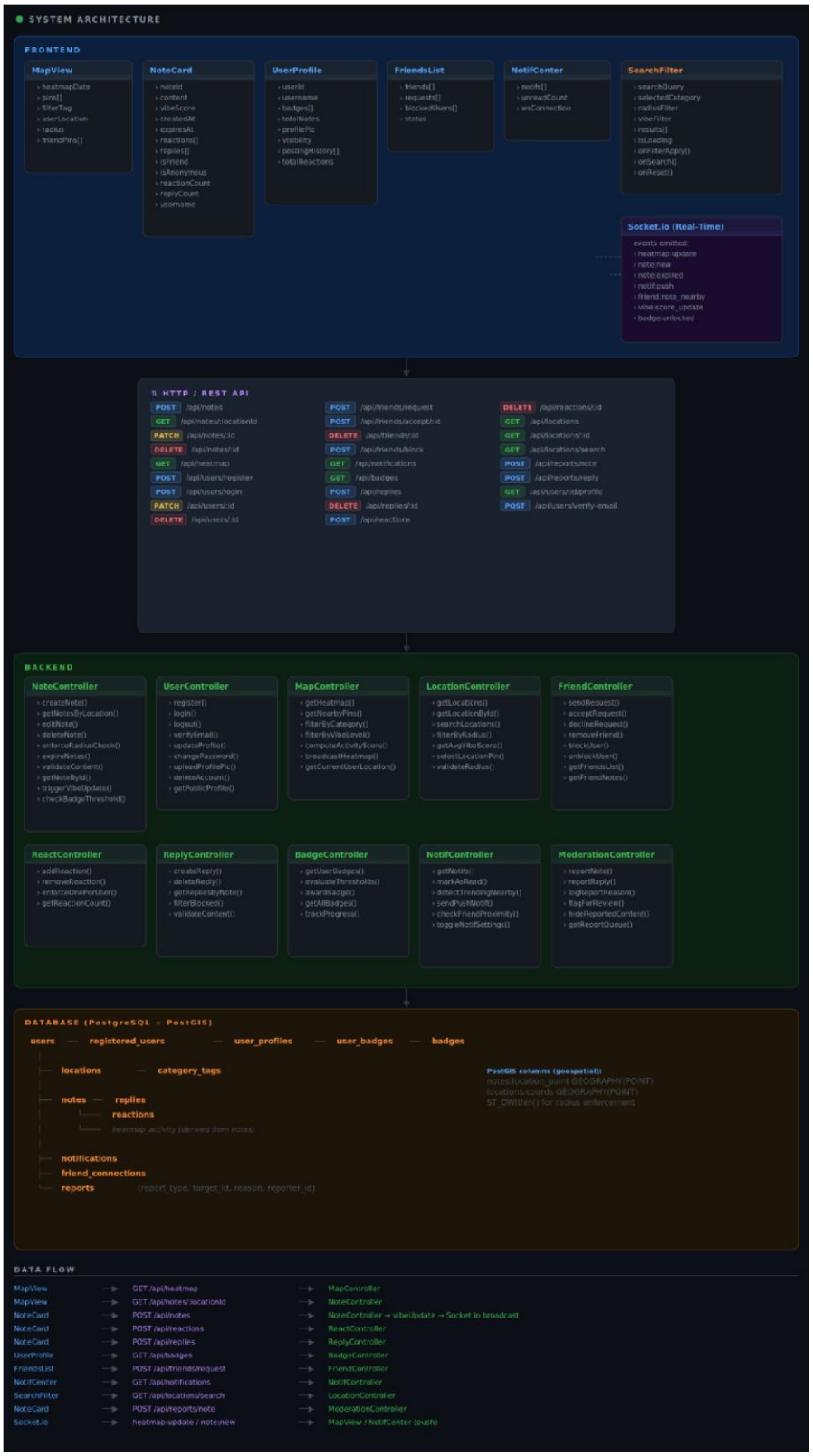
ERD



Scalability Diagram

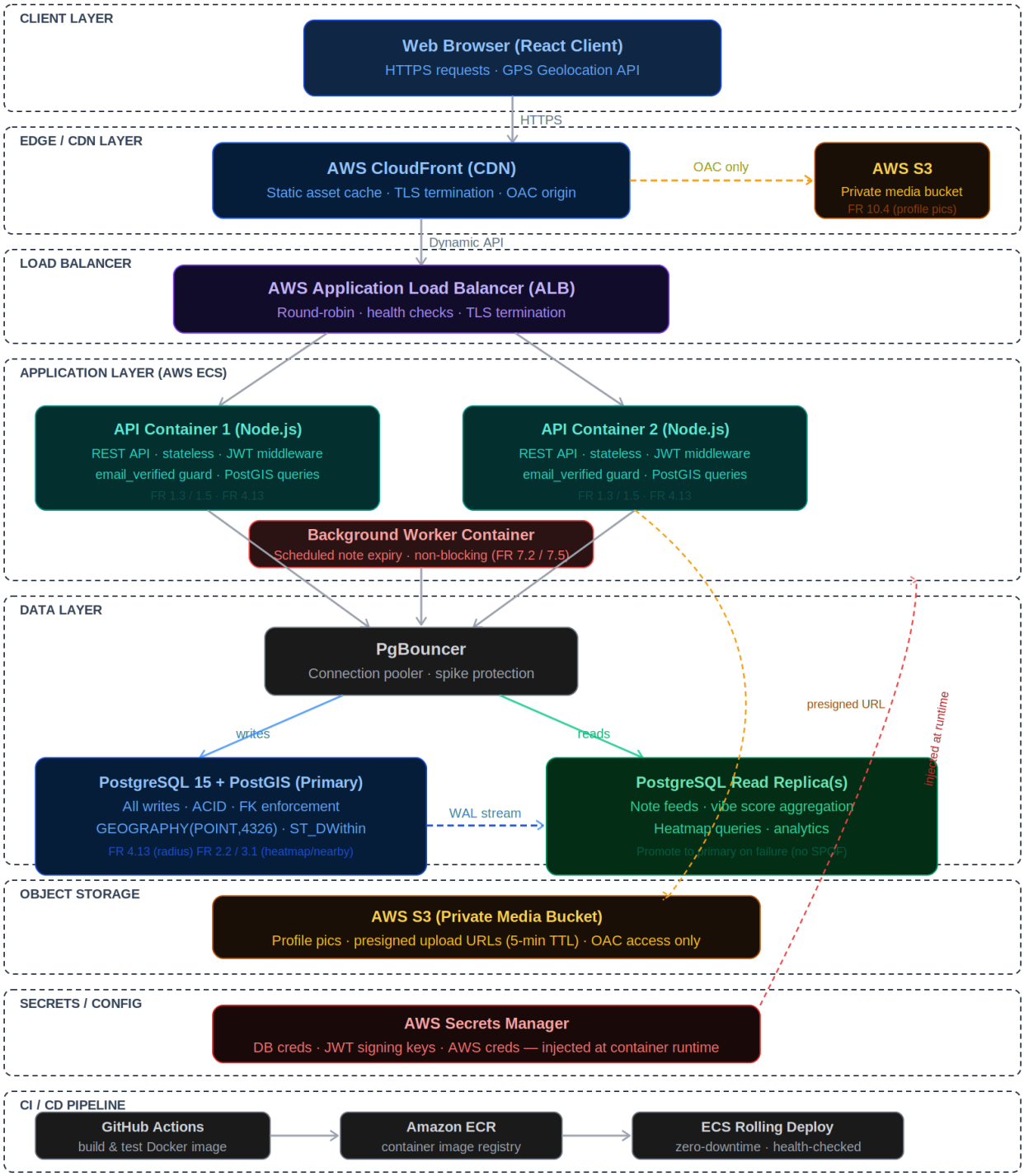

VIBE CHECK — SCALABILITY DIAGRAM
Web Browser
(React Client)
Static assets
API calls
CDN
Static Asset Delivery
AWS Route 53
DNS Routing
AWS Elastic Load Balancer (ALB)
Round-robin routing
More users = add more App Servers
App Server 1
(EC2 / Node.js)
POST /api/notes
GET /api/heatmap
POST /api/users
GET /api/friends
GET /api/notifications
Cron job runs here (1 server only)
App Server 2
(EC2 / Node.js)
POST /api/notes
GET /api/heatmap
POST /api/users
GET /api/friends
GET /api/notifications
All servers share the same:
PostgreSQL Primary (RDS)
All writes
ACID compliant
FK enforcement
PostGIS ext.
Radius queries
↓ WAL stream
Redis Cache
(Heatmap / App Cache)
2 min TTL
activity_score
avg_vibe_score
Replaces in-memory heatmap cache
★ Shared across all app servers
Redis Session Store
(Auth / Sessions)
Auth tokens
Session data
TTL-based expiry
Replaces in-memory session store
★ Shared across all app servers
Expiry Job (Cron)
DELETE expired notes
Runs every 15 min
Runs on 1 server only
WAL stream
PostgreSQL Replica (RDS)
All reads
Heatmap queries
Analytics
Backup and Recovery
Nightly pg_dump → Compressed → S3 upload
Failover: promote replica manually if primary fails
How Horizontal Scaling Works Here
More users = add more App Servers
Load Balancer automatically distributes requests across them
All servers share the same:
- PostgreSQL Primary (writes) - PostgreSQL Replica (reads)
- Redis Cache (heatmap) - Redis Session Store (auth)
- CDN (static assets, served independently)
* Stateless app servers = scale freely | Redis replaces per-server in-memory stores | CDN offloads static traffic

UML Diagram



Network and Deployment Design

Network Diagram



High Level APIs and Algorithms

Internal Backend APIs

The non-trivial endpoints below contain significant business logic beyond basic CRUD.

Endpoint	Method	Description
/notes	POST	Create a note. Enforces geographic radius before persisting. Returns descriptive error on failure
/notes/:id	PUT / DELETE	Edit or delete the authenticated user's own note. Triggers vibe score recalculation on delete.
/locations/:id/vibe	GET	Returns the aggregate vibe score from non-expired notes only. Returns fallback message when note count is below minimum threshold.
/heatmap	POST	Returns activity_score per geographic grid cell within a rolling 2-hour window. Supports category filter param.
/notifications/trigger	POST (internal)	Called by background worker. Evaluates trending threshold and dispatches push notifications to eligible nearby users

External Libraries/APIs

1.)Leaflet.js + OpenStreetMap

Purpose: Core interactive map rendering, displays location pins, heatmap overlay, zoom/pan, and radius circles.
Used in: Heatmap view, location pin display, friend indicator icons

2.) Socket.io

Purpose: Real-time bi-directional WebSocket communication between client and server. Used in: Live heatmap updates, instant vibe score refresh, friend activity notifications.

3.)PostGIS (PostgreSQL Extension)

Purpose: Geospatial queries on top of PostgreSQL.
Used in: Radius enforcement for note posting, proximity-based location search , heatmap cell bucketing.

Non-Trivial Algorithms

1.) Aggregate Vibe Score Calculation

Computes the current atmosphere rating for a location from only active (non-expired) notes. Vibe Level Mapping

Descriptive Label

Descriptive Label	Numeric Value
Dead	1
Quiet	2
Moderate	3
Busy	4
Buzzing	5

Process

Query all non-expired notes at the location (WHERE expires_at > NOW())
If note count is below the minimum threshold, return a fallback message instead of a score.
Compute AVG(numeric_value). Round to nearest integer, map back to the descriptive label for display
Recalculate is triggered by: new note posted, note deleted, or background expiry job.

2.)Location-Locked Posting Radius Enforcement

Ensures users can only submit notes from within a defined geographic radius of a location. This is one of Vibe Check's core trust mechanisms (96.2% of surveyed users indicated location-lock increases trust).

Process

Client submits note with user GPS coordinates and target location_id
Server fetches the location's stored latitude/longitude from the database.
Computes Haversine distance between user coordinates and location coordinates.
If distance exceeds the defined radius, reject the request with a descriptive error.
If within radius, persist the note. Can also be implemented using PostGIS ST_DWithin() for efficiency.

3) Note Expiration and Cascade Effects
Notes auto-delete 2–4 hours after creation. Expiration is not just a deletion event - it triggers cascading recalculations.
Process
A background worker (cron job or PostgreSQL scheduled task) runs periodically to identify notes where expires_at <= NOW().
For each expired note, the worker: (a) deletes or soft-deletes the note record, (b) triggers a vibe score recalculation for the location, (c) invalidates the heatmap snapshot for affected grid cells.
Socket.io emits a vibe_updated event to any clients currently viewing the affected location.

4) Heatmap Activity Computation
Computes a visual intensity score per geographic cell from recent note activity.
Input
All non-expired notes within the user's visible map bounds.
Rolling time window (default: 2 hours)
Process
Divide the visible map area into a uniform grid of cells (using latitude/longitude bounding boxes or a hexagonal grid library such as H3).
For each cell, count non-expired notes whose coordinates fall within the cell boundary (PostGIS ST_Within).
Map the count to a heat intensity value and render via Leaflet.js heat layer.
Store a computed_at snapshot to avoid recomputing on every map pan. Invalidate when a note is added or expires.

Key Project Risks

Skills Risks

Risk: Team lacks expertise in required technologies (e.g., a new framework, cloud platform, or Database. **Mitigation:** Conduct a skills gap assessment early or schedule targeted training. Pair junior devs with senior leads on high-risk modules.

Schedule Risks

Risk: Underestimated task complexity, scope creep, or dependency delays (e.g., waiting on third-party APIs or another team's deliverable). **Mitigation:** Use time-buffered sprint planning (add 20–30% buffer to estimates). Define a formal change request process to control scope. Identify critical path dependencies upfront and monitor them weekly.

Technology Risks

Risk: Chosen technology proves unstable, deprecated, or incompatible with existing systems mid-project. **Mitigation:** Run a time-boxed technical spike/proof-of-concept before committing. Prefer well-supported, battle-tested libraries. Document fallback technology options before build begins.

Teamwork Risks

Risk: Poor communication, unclear ownership, or interpersonal conflict causing bottlenecks or duplicated work. **Mitigation:** Establish a RACI matrix (Responsible, Accountable, Consulted, Informed) at kickoff. Hold regular standups and retrospectives. Use a shared project tracker (Jira, Linear, etc.) to keep ownership visible.

Legal & Compliance Risks

Risk: Use of open-source libraries with incompatible licenses (e.g., GPL in a commercial product), data privacy violations (GDPR, CCPA), or IP ownership ambiguity with contractors. **Mitigation:** Audit all third-party dependencies for license compatibility early. Involve legal counsel for data handling design decisions. Ensure contractor agreements include clear IP assignment clauses.

Project Management

https://arkha.atlassian.net/jira/software/projects/KAN/boards/1.

Figma Wire frames

https://www.figma.com/design/LYGQ4kU3C2JWDyIqQBACYH/Vibe-Check-Wireframes?node-id=0-1

Design Patterns

1. Singleton - FR 3.1 (Heatmap / shared map) and FR 3.3 (Zoom in/out)

Singleton = One instance sharing
All components of the app must share the same map instance. The sidebar, heatmap layer, markers, and zoom controls all need to read from and write to one single Leaflet map, not create their own. If two map instances exist, a marker placed by the sidebar would not show on the heatmap layer.

Uber app example: All components must share the same map → being able to locate all vehicles, being able to locate the customer.

Right now in Map.jsx we are using useRef to hold the map, which is already acting like a Singleton within the component, but as the app grows (heatmap layer, note pins, vibe overlays), we need a proper shared instance.

FR mapping:

- FR 3.1 - The heatmap layer calls MapInstance.getInstance().addHeatLayer(data) to overlay activity on the same map the user is viewing.
- FR 3.3 - Zoom controls call MapInstance.getInstance().getZoom() and the map is the same shared object.

```
+---------------------------------+
|         MapInstance             |
+---------------------------------+
| - instance: MapInstance         |
| - leafletMap: L.Map             |
| - heatLayer: L.HeatLayer        |
+---------------------------------+
| + getInstance(): MapInstance|
| + getMap(): L.Map               |
| + addHeatLayer(data): void   | |
+ flyTo(lat, lng, zoom): void  |
| + getZoom(): int                |
+---------------------------------+
```

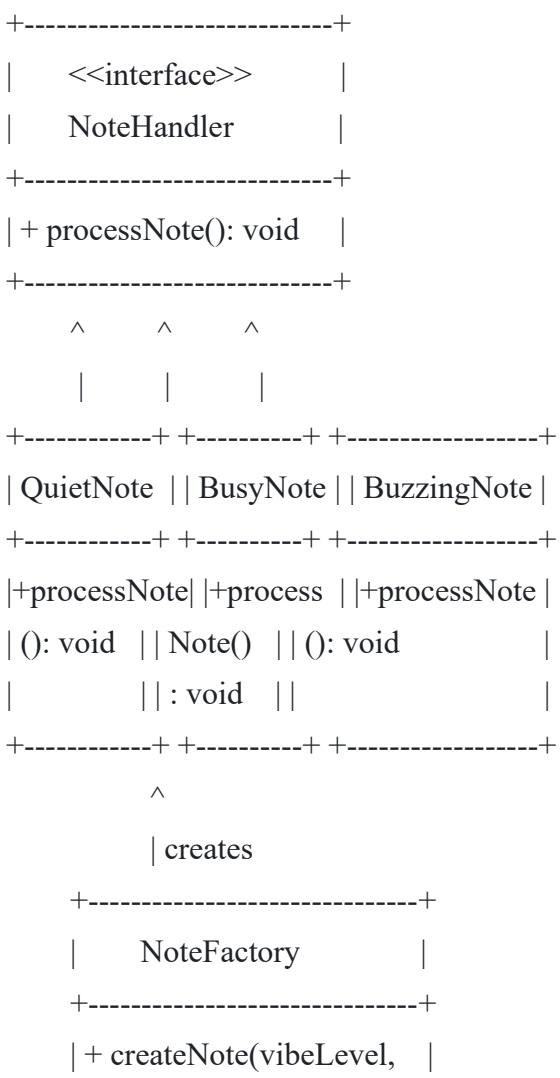
2. Factory - FR 4.9/4.10 (Post a note) and FR 7.1 (Select a vibe level)

Factory = Decides what to create
When a user creates a note (FR 4.9, 4.10), they select a vibe level from Dead/Quiet/Moderate/Busy/Buzzing (FR 7.1). The system needs to decide what type of note object to create based on that vibe level. Each vibe level has different display rules (color, icon, weight in the heatmap score from heatmap_activity). A NoteFactory decides which concrete note type to instantiate.

Amazon example: Orders need to be designed as digital product orders OR physical product orders.
Our version is: "Notes need to be created as Dead notes, Quiet notes, Moderate notes, Busy notes, or Buzzing notes."

FR mapping:

- FR 4.9 / 4.10 - User posts a note with text content. The factory creates the right note type.
- FR 7.1 - The vibe level selection (dead, quiet, moderate, busy, buzzing from our vibe_level_enum in schema.sql) is the input that tells the factory which concrete note to create.
- FR 4.14 - If the factory cannot create the note (missing content, invalid vibe level), it throws a descriptive error message.



```
|   content, ...): NoteHandler|
+-----------------------------+
```

3. Builder - FR 4.15 (Note card display) and FR 4.17/4.18 (Username and timestamp)

Builder = Builds objects step by step

FR 4.15 says note cards display: vibe level, note content, timestamp, reaction count, and reply count. FR 4.17 adds the username or anonymous handle. FR 4.18 adds time elapsed. A note card is a complex object that gets built step by step, not all at once. Some fields are optional (anonymous vs. username), some are computed (time elapsed, reaction count).

Amazon example: Every product needs to be created step by step and not directly.

The NoteCardBuilder uses method chaining (each setter returns this) and a final build() that returns the finished NoteCard object, like the ProductBuilder class example

FR mapping:

- FR 4.15 - The builder assembles vibe level, content, timestamp, reaction count, reply count step by step.
- FR 4.17 - setUsername() handles either the real username or the anonymous handle depending on is_anonymous from our notes table.
- FR 4.18 - setTimeElapsed() computes "2h ago", "5m ago" from the created_at column.
- FR 7.4 - setVibeLevel() converts the enum to a descriptive label ("Buzzing" not "5").

```
+-----------------+      +-------------------------------------------------+
|   NoteCard      |      |         NoteCardBuilder                         |
+-----------------+      +-------------------------------------------------+
| - content       |      | + setContent(...): NoteCardBuilder              |
| - vibeLevel     |<---- | + setVibeLevel(...): NoteCardBuilder            |
| - vibeLabel     |      | + setUsername(...): NoteCardBuilder             | |
- username        |      | + setTimeElapsed(...): NoteCardBuilder          |
| - timeElapsed   |      | + setReactionCount(...):NoteCardBuilder         |
| - reactionCount |      | + setReplyCount(...): NoteCardBuilder           |
| - replyCount    |      | + build(): NoteCard                             |
| - isAnonymous   |      +-------------------------------------------------+
+-----------------+
```

4. Prototype - FR 4.9 (Post a note at a specific location) and FR 7.2 (Vibe score from non-expired notes)

Prototype = Copy an object

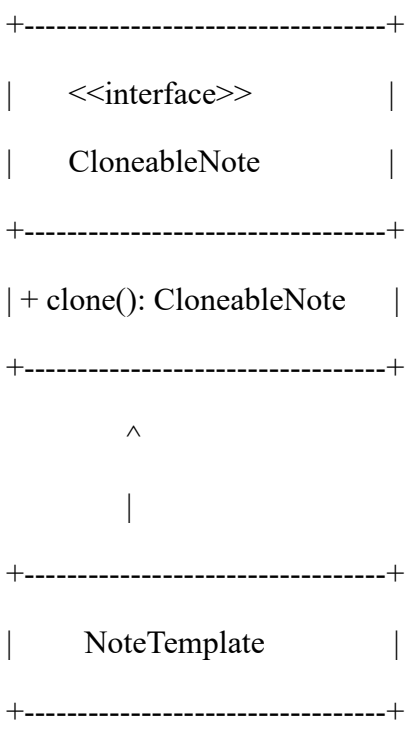
When a user wants to post a new note at a location where notes already exist, the system can clone the location's default note template (pre-filled with location_id, radius_meters, the location's category_tags from our locations table) instead of building the object from scratch every time.

Amazon example: All product listings should support cloning.

Also useful for FR 7.2: when computing vibe scores, we query all non-expired notes for a location. If the same query result set is needed by both the vibe score display and the heatmap, we clone the note objects rather than hitting the database twice.

FR mapping:

- FR 4.9 - User posts at a location. The system clones a NoteTemplate pre-loaded with that location's data, then the user just fills in content and vibe level.
- FR 4.13 - The cloned template already has radiusMeters baked in from the locations table, so the geo-radius check is part of the template.
- FR 7.2 - Cloned note data can be reused across vibe score calculation and heatmap rendering without re-querying.



| - locationId |

| - locationName |

| - radiusMeters |

| - categoryTags |

| - defaultVibeLevel |

+---------------------------------+

| + clone(): NoteTemplate |

| + setContent(content): void |

| + setVibeLevel(level): void |

| + setUserId(userId): void |

+---------------------------------+

Team Contributions

- Harry Kakadiya (Team Lead)
  Version 1 Contributions:
  - Reviewed and merged PRs to master
  - Wrote Team Contributions section
  - Coordinated task distribution across all team members

- Kaitlyn Ashby (GitHub Master, Technical Writer) -
  Version 1 Contributions:
  - Added documentation and formatted sections in shared doc
  - Managed develop branch and opened PR to main
  - Created UML design pattern diagrams for the document
  - Added Figma screenshots to document

- Rahul Srinath (Software Architect) -

Version 1 Contributions: -

- Wrote and finalized High Level Functional Requirements section
- Managed EC2 instance and server configuration

- Amulya Sagi (Backend Lead)
  Version 1 Contributions:
  Created Figma wireframes showing user flows and screen transitions with navigation hyperlinks

- Aljhay Soriano (Scrum Master)
  Version 1 Contributions:
  - Helped with Figma wireframes
  - Added navigation hyperlinks to Figma wireframes connecting all pages
  - Submitted PR #30 for review and merge

Member	**Score**
Harry	5/5
Kaitlyn	5/5
Rahul	5/5
Amulya	5/5
Aljhay	5/5